강화학습 Day01 hw1 보고서

작성자: Doyeon Park

작성일: 2026.10.29

목차

1. 코드분석
2. 실험결과
3. 오류 사례
4. 결론 및 개선할 점
5. 실행 방법
6. 참고 자료 및 GPT 사용 내역

1. 코드 분석

1.1. 실행 환경 및 데이터 준비

VS Code 에서 Python 가상환경을 구성하고 CPU 용 PyTorch 를 사용하였다.

항목	설정
Python	3.12.3
PyTorch	2.14.1+cpu
torchvision	0.29.1+cpu
계산 장치	CPU
시각화	Matplotlib

MNIST 는 28×28 크기의 흑백 손글씨 이미지와 정답 라벨로 구성된다. 원본 학습 데이터 60,000 개를 학습용 55,000 개와 검증용 5,000 개로 나누고, 별도의 테스트 데이터 10,000 개를 사용하였다.

학습용 데이터는 가중치와 편향을 갱신하는 데 사용하였다. 검증용 데이터는 학습 도중 성능 변화를 확인하는 데 사용하였으며, 테스트 데이터는 각 실험 종료 후 성능을 측정하는 데 사용하였다. 검증과 테스트에서는 역전파나 파라미터 갱신을 수행하지 않는다.

transforms.ToTensor()를 적용하여 이미지를 [1, 28, 28] 형태의 텐서로 변환하고, 픽셀값을 0~255 정수에서 0~1 실수로 조정하였다. 학습 데이터는 최대 64 개씩 묶고 매 epoch 마다 순서를 섞었다. 검증과 테스트는 최대 256 개씩 처리하였다.

난수 시드는 0 으로 설정하였고, 데이터 분할에도 별도의 시드 0 을 사용하였다. 이는 재현성을 높이기 위한 설정이며, 실행 환경에 관계없이 모든 계산 결과가 완전히 동일함을 보장하는 것은 아니다.

1.2. 모델 구조와 학습 설정

모델은 다음 순서로 구성되어있다.

입력 이미지 [B, 1, 28, 28]
→ Flatten [B, 784]
→ Linear(784, 128)
→ ReLU
→ Linear(128, 10)
→ 숫자별 점수 [B, 10]
B 는 한 묶음에 들어 있는 이미지 수이다.

Flatten 은 각 이미지의 픽셀을 784 개의 값으로 펼친다. 첫 번째 Linear 는 입력 784 개로부터 은닉값 128 개를 계산한다. ReLU 는 음수를 0 으로 바꾸고 양수는 유지하여 비선형성을 추가한다. 마지막 Linear 는 숫자 0~9 에 대한 점수 10 개를 출력한다. 이 점수를 logits 라고 하며, 가장 높은 점수의 인덱스를 예측 숫자로 선택한다.

항목	설정

손실 함수	CrossEntropyLoss
옵티마이저	Adam
학습률	0.001
학습 횟수	5, 10, 20 epoch 비교
학습 배치 크기	64

모델 구조와 학습률은 교육 자료의 기본 설정을 사용하였다.

1.3. 학습 및 평가

학습에서는 각 미니배치마다 다음 과정을 반복하였다.

```
optimizer.zero_grad()
logits = model(images)
loss = criterion(logits, labels)
loss.backward()
optimizer.step()
```

optimizer.zero_grad()는 이전 묶음의 기울기를 초기화한다. PyTorch 는 기울기를 누적하므로, 현재 묶음의 기울기만 반영하기 위해 필요하다.

model(images)는 현재 가중치와 편향으로 순전파를 수행한다. criterion(logits, labels)는 예측 점수와 정답을 비교하여 묶음의 평균 손실을 계산한다. CrossEntropyLoss 에는 Softmax 를 따로 적용하지 않고 logits 를 직접 전달하였다.

loss.backward()는 각 파라미터에 대한 손실의 기울기를 계산한다. 이 단계에서는 파라미터가 바뀌지 않는다. 이후 optimizer.step()이 계산된 기울기를 이용해 Adam 규칙으로 가중치와 편향을 갱신한다.

학습 손실은 각 묶음의 평균 손실에 실제 이미지 수를 곱하여 누적한 뒤, 전체 학습 이미지 수로 나누어 계산하였다. 이 방법은 마지막 묶음의 크기가 작더라도 이미지당 평균을 올바르게 계산한다.

평가에서는 model.eval()로 평가 모드를 설정하고 torch.no_grad()로 역전파용 연산 기록을 중단하였다. 예측과 정답을 비교하여 평균 손실과 정확도를 계산했으며, 가중치는 변경하지 않았다.

2. 실험 결과

2.1 최종 결과

새로 실행한 5·10·20 epoch 실험의 최종 결과는 다음과 같다. 각 행의 학습·검증 값은 해당 실험의 마지막 epoch 에서 측정한 값이다.

학습 횟수	학습 손실	검증 손실	검증 정확도	테스트 손실	테스트 정확도
5	0.0669	0.0958	97.04%	0.0828	97.48%
10	0.0243	0.0883	97.24%	0.0801	97.65%
20	0.0070	0.1054	97.50%	0.0933	97.89%

테스트 데이터 10,000 개를 기준으로 5 epoch 모델은 252 개, 10 epoch 모델은 235 개, 20 epoch 모델은 211 개를 틀렸다.

2.2 Epoch 별 학습·검증 변화

아래 표는 20 epoch 실험의 전체 학습 기록이다. 5·10 epoch 실험은 각각 별도로 실행하였으며 사진으로 첨부하였다.

Epoch	학습 손실	검증 손실	검증 정확도
1	0.3522	0.2036	94.14%
2	0.1606	0.1452	95.46%
3	0.1121	0.1164	96.64%
4	0.0851	0.1064	96.86%
5	0.0669	0.0958	97.04%
6	0.0551	0.0853	97.54%
7	0.0441	0.0883	97.38%
8	0.0360	0.0869	97.36%
9	0.0302	0.0833	97.76%
10	0.0243	0.0903	97.20%
11	0.0213	0.0844	97.64%
12	0.0169	0.0905	97.38%
13	0.0149	0.0870	97.58%
14	0.0128	0.0873	97.54%
15	0.0109	0.0895	97.72%
16	0.0080	0.0908	97.54%
17	0.0081	0.0917	97.68%
18	0.0057	0.0925	97.68%
19	0.0057	0.1024	97.44%
20	0.0070	0.1054	97.50%

2.3 결과 해석

학습 초기에는 학습 손실과 검증 손실이 함께 감소하였다. 이는 학습을 통해 손글씨를 구분하는 능력이 개선되었음을 알수 있었다.

20 epoch 실험에서 검증 손실은 9 번째 epoch 에 0.0833 으로 가장 낮았다. 이후 학습 손실은 전반적으로 더 감소했지만 검증 손실은 변동하며 최종적으로 0.1054 까지 증가하였다. 학습 데이터에 대한 적합은 개선되지만 검증 데이터에 대한 개선이 지속되지 않는 과적합 징후로 볼 수 있다.

20 epoch 모델의 테스트 정확도는 세 실험 중 가장 높았지만, 테스트 손실은 10 epoch 모델보다 높았다. 정확도는 가장 높은 점수를 받은 클래스가 정답인지 확인하는 반면, Cross Entropy 는 정답에 부여한 확률도 반영한다. 따라서 더 많은 문제를 맞히더라도 일부 오답에 높은 확신을 보이면 손실이 커질 수 있다.

3. 오분류 사례

테스트 데이터에서 예측 숫자와 정답이 다른 이미지를 찾아, 순서상 처음 발견된 9 장을 저장하였다. 그림은 png 사진으로 첨부하였다. 그림의 True 는 정답 라벨이고 Pred 는 모델의 예측이다.

정답 → 예측	관찰한 특징과 가능한 혼동 원인
4 → 9	위쪽 획이 굽어 연결되고 오른쪽 세로획이 길어, 9 와 비슷한 형태로 인식했을 가능성이 있다.
4 → 6	아래쪽 획이 복잡하게 연결된 특이한 필기 형태로, 일반적인 4 의 형태와 차이가 있다.
8 → 4	위쪽 고리가 열려 있고 획이 교차하여 4 와 유사하게 보일 수 있다.
2 → 7	위쪽 가로획과 길게 내려오는 획이 두드러져 7 로 혼동했을 가능성이 있다.
5 → 3	위쪽 꺾임이 뚜렷하지 않고 오른쪽으로 굽은 형태가 나타나 3 과 혼동했을 가능성이 있다.
6 → 0	큰 고리 형태가 두드러지고 위쪽 획이 상대적으로 약해 0 으로 인식했을 가능성이 있다.
8 → 0	두 고리의 경계와 외곽 형태를 충분히 구분하지 못했을 가능성이 있다.
8 → 2	위쪽 고리가 열리고 아래쪽 고리가 두드러지는 특이한 필기 형태가 나타난다.
2 → 1	폭이 좁고 세로 방향의 획이 두드러져 1 로 혼동했을 가능성이 있다.

4. 결론 및 개선할 점

본 실습을 통해 신경망 학습(예측, 손실 계산, 기울기 계산, 파라미터 갱신을 반복)에 대해 이해할 수 있었다.

학습 횟수를 늘리면 학습 손실은 전반적으로 줄었지만 검증 손실이 계속 개선되지는 않았다. 따라서 학습 손실만으로 모델의 성능을 판단하지 않고, 검증 결과와 함께 확인해야 한다는 것을 알 수 있었다.

또한 최대한 성능을 뽑기 위해 검증 손실이 가장 낮은 시점의 모델을 저장하거나 조기 종료를 적용하는 방법을 사용할 수 있고, 또한 여러 시드로 반복 실험하고, 이미지의 공간적 특징을 활용하는 CNN 이나 데이터 증강을 비교하는 방법또한 존제한다.

5. 실행 방법

실습 폴더에 Python 가상환경을 만들고 필요한 라이브러리를 설치한다.

```
python3 -m venv .venv
.venv/bin/python -m pip install torch torchvision --index-url https://download.pytorch.org/whl/cpu
.venv/bin/python -m pip install matplotlib
```

VS Code 에서 .venv/bin/python 을 인터프리터로 선택한 뒤 코드를 실행한다. 현재 파일명 기준 터미널 실행 방법은 다음과 같다.

```
.venv/bin/python "import torch.py"
```

첫 실행 시 MNIST 데이터가 data 폴더에 다운로드된다. 실행 후 epoch 별 학습·검증 결과와 최종 테스트 결과가 출력되고, 오분류 이미지는 wrong_predictions.png 에 저장된다. 학습 횟수는 코드의 epochs 값을 변경하여 설정하였다.

6. 참고 자료 및 GPT 사용 내역

기본 구현은 교육 자료의 예시 코드를 바탕으로 하였다.

GPT 는 다음 목적으로 사용하였다.

사용 부분	사용 이유
텐서, 기울기, 역전파, 옵티마이저 설명	신경망 학습 원리 이해
Python 문법과 코드 설명	import, 함수, 반복문, 들여쓰기 이해
가상환경 및 설치 오류 해결	PyTorch 실행 환경 구성
epoch 별 검증·최종 테스트 코드 추가	학습 과정과 최종 성능 확인
오분류 이미지 시각화 코드	틀린 사례를 찾기 위함